



**INSTITUTE OF INFORMATION TECHNOLOGY
JAHANGIRNAGAR UNIVERSITY
SAVAR, DHAKA-1342**

OPERATING SYSTEM LAB

Course Code: ICT-3102

LAB REPORT NO: 05

Lab Title:

**Working with Users, Groups, and File Permissions
(User and Group Management)**

Submitted by:

Md. Shaon Khan

Student ID: 1984

52nd Batch | Session: 2022-2023

Supervised by:

Mehrin Anannya

Assistant Professor

Institute of Information Technology

Jahangirnagar University

Submission Date: May 26, 2026

1. Objective

The primary objective of this laboratory session is to develop a working understanding of Linux user and group administration from the perspective of a system administrator. The lab covers the complete lifecycle of a user account, from creation to deletion, and explores the mechanisms by which Linux enforces access control through file ownership and permission bits.

Specifically, the following goals guided the experiment:

- (i) To understand the structure and role of the Linux multi-user architecture, including how user and group information is stored in system configuration files.
- (ii) To learn the commands required to create, configure, switch between, and remove user accounts on a Linux system.
- (iii) To distinguish between the behavior of `su` and `su -` when switching user sessions, and to understand the consequence of each on the shell environment.
- (iv) To practice group creation, group membership assignment, and the use of the `usermod` command for assigning supplementary groups to an existing user.
- (v) To modify file ownership and group ownership using the `chown` and `chgrp` commands, and to understand the access constraints governing these operations.
- (vi) To apply special permission bits, specifically the Set Group ID (SGID) flag, to directories using the `chmod` command, and to observe its effect on the permission string displayed by `ls -l`.
- (vii) To understand the security and administrative significance of each concept practiced, including privilege separation, permission enforcement, and collaborative workspace management.

2. Introduction and Theory

2.1 Linux as a Multi-User Operating System

Linux is a multi-user and multi-tasking operating system derived from the UNIX tradition. Its design inherently supports simultaneous access by multiple users, each of whom is given an isolated environment within which their files, processes, and credentials are protected from unauthorized access by other users. This isolation is enforced at the kernel level through a system of user identifiers (UIDs) and group identifiers (GIDs), which are associated with every process and every file on the filesystem.

Unlike single-user operating systems, Linux maintains strict boundaries between accounts. A standard user can read, write, and execute files only to the extent permitted by the file permission bits and ownership attributes. This arrangement ensures that a compromised or misbehaving user account cannot affect system-wide resources without elevated privileges. The superuser account, traditionally named root, holds UID 0 and is exempt from these permission checks, allowing it to perform unrestricted administrative operations.

Administrative tasks such as creating users, modifying groups, and changing file ownership are generally restricted to the root account or to users who have been granted the ability to execute specific commands with elevated privilege via the sudo mechanism. This separation of ordinary and privileged operations is a fundamental security design principle in Linux.

2.2 User Account Databases

Linux stores user account information in two primary configuration files: `/etc/passwd` and `/etc/shadow`. The `/etc/passwd` file contains one entry per account and is world-readable. Each entry records the username, a placeholder for the password field, the UID, the primary GID, the GECOS field (which holds optional personal information such as full name, room number, and phone numbers), the home directory path, and the default login shell. The world-readability of this file allows programs to map UIDs to usernames without requiring elevated privileges.

Since storing actual password hashes in a world-readable file would be a security vulnerability, Linux separates authentication credentials into `/etc/shadow`. This file is readable only by the root user and the shadow group. It stores the hashed password for each account, along with metadata such as the last password change date, minimum and maximum password age, and account expiration settings. This division between public account metadata and private authentication data is known as shadow password support and is enabled by default on all modern Linux distributions.

Group information is stored in `/etc/group`, which lists each group by name, its GID, and the usernames of its supplementary members. A user always has a primary group, whose GID is recorded in `/etc/passwd`,

and may belong to any number of supplementary groups recorded in `/etc/group`. Supplementary group membership determines what group-owned resources the user can access in addition to those accessible through the primary group.

2.3 User Switching: su versus su -

The `su` command, which stands for switch user, allows a logged-in user to assume the identity of another account by providing that account's password. However, the behavior of `su` differs meaningfully depending on whether it is invoked with the hyphen flag or without it.

When executed as `su username` without the hyphen, Linux spawns a new shell under the target user's identity but preserves the calling user's environment. This means that environment variables such as `HOME`, `PATH`, and `SHELL` retain their values from the original session, and the working directory remains wherever the user was before invoking the command. In practical terms, this can cause unexpected behavior because the new session does not properly initialize the target user's environment.

When executed as `su - username`, Linux invokes a login shell for the target user. This means the system reads the target user's shell initialization files, such as `/etc/profile` and `~/.bashrc`, sets `HOME` to the target user's home directory, updates `PATH` and other environment variables appropriately, and changes the working directory to the target user's home directory. This behavior is equivalent to a fresh login as that user. In administrative practice, `su -` is the preferred form because it guarantees a properly initialized environment, avoids inheriting environment variables from the calling session, and accurately reflects what the target user would see upon logging in directly.

2.4 Special Permission Bits: SUID and SGID

Linux supports three special permission bits beyond the standard read, write, and execute triplets: the Set User ID (SUID) bit, the Set Group ID (SGID) bit, and the sticky bit. The SUID and SGID bits are particularly relevant in the context of privilege management and collaborative file access.

The Set User ID (SUID) bit, when applied to an executable file, causes the process launched by that executable to run with the effective UID of the file's owner rather than the UID of the user who invoked it. A well-known example is the `/usr/bin/passwd` program, which must write to `/etc/shadow` despite being executed by an ordinary user. Since `/etc/shadow` is readable only by root, the `passwd` binary carries the SUID bit and runs as root regardless of who invokes it. This mechanism allows controlled temporary elevation of privilege without granting the user unrestricted root access.

The Set Group ID (SGID) bit has a related but distinct function. When applied to an executable, it causes the process to run with the effective GID of the file's group rather than the calling user's primary group. When SGID is applied to a directory, it has an even more useful effect: all files and subdirectories created

within that directory automatically inherit the group ownership of the directory itself, rather than the primary group of the user who created them. This behavior is particularly valuable in shared or collaborative directories where multiple users from the same group are expected to contribute files that must remain accessible to all members of that group. Without SGID, each file would inherit the creator's primary group, making it potentially inaccessible to other group members who do not share that primary group.

In the permission string output of `ls -l`, the presence of SUID or SGID is indicated by the letter `s` in the position normally occupied by the execute (`x`) bit. If the execute bit was already set, the permission displays as `s`. If it was not set, it displays as `S`, indicating that the special bit is present but the ordinary execute permission is absent.

3. Commands and Utilities Used

Command	Purpose	Description
getent passwd	Display user accounts	Retrieves entries from the Name Service Switch databases, including /etc/passwd, and prints complete account information for all users on the system.
compgen -u	List usernames only	A shell built-in command that generates a plain list of all usernames known to the system, without additional fields, making the output easier to scan than getent passwd.
sudo adduser <user>	Create a new user account	Creates a new user account interactively, assigning a UID, creating a primary group, initializing the home directory from /etc/skel, and prompting for the password and GECOS information.
sudo passwd <user>	Reset a user's password	Allows a privileged user to set or change the password of any account directly, bypassing the interactive adduser password prompt. Useful for setting a known password after account creation.
su - <user>	Switch to user with full login environment	Opens a login shell as the specified user, fully initializing that user's environment variables, home directory, and shell configuration files, as if the user had logged in directly.
sudo userdel -r <user>	Delete a user and their home directory	Removes the specified user account from the system. The -r flag additionally removes the user's home directory and mail spool, ensuring that no orphaned files remain after deletion.
sudo groupadd <group>	Create a new group	Creates a new group with the specified name. The kernel assigns the next available GID automatically, and the group is registered in /etc/group.
sudo tail /etc/group	View recently added groups	Displays the last ten lines of the /etc/group file, making it convenient to verify newly created groups and confirm their assigned GID values.
groups [user]	List group memberships	Prints the names of all groups to which the specified user belongs, including the primary group and any supplementary groups. When called without an argument, it reports the current user's groups.
sudo usermod -aG <groups> <user>	Add user to supplementary groups	Modifies an existing user account to add them to the specified supplementary groups. The -a flag appends without removing existing memberships; the -G flag specifies the comma-separated list of target groups.

Command	Purpose	Description
sudo chown <user> <file>	Change file owner	Transfers ownership of the specified file to the specified user. Only root or a user with sudo privileges can transfer file ownership to another account.
sudo chgrp <group> <file>	Change file group ownership	Changes the group associated with the specified file to the given group name. An ordinary user may only change a file's group if they own the file and belong to the target group; sudo is required otherwise.
chmod ug+s <dir>	Set SUID and SGID bits	Applies the Set User ID and Set Group ID special permission bits to the specified directory or file. For a directory, SGID causes all newly created files within it to inherit the directory's group ownership automatically.

4. Step-by-Step Experimental Procedure

4.1 Exploring Existing System Accounts

The experiment began by examining the set of user accounts already present on the system. The first command used was `getent passwd`, which queries the Name Service Switch database and prints the full contents of `/etc/passwd` in a structured, colon-delimited format. Each line of the output corresponds to one account and contains the username, a placeholder `x` in the password field indicating that the actual hash is stored in `/etc/shadow`, the numeric UID, the numeric GID, the GECOS field, the home directory, and the default shell.

The output showed a large number of system accounts, including well-known service accounts such as `root`, `daemon`, `bin`, `sys`, `sync`, `games`, `man`, `lp`, `mail`, `news`, `uucp`, `proxy`, `www-data`, `backup`, and many others maintained by the Ubuntu package management system. System accounts generally have UIDs below 1000 and are assigned `/usr/sbin/nologin` or `/bin/false` as their shell, which prevents interactive login. The only human account visible in the output was `shaonkhan`, assigned UID 1002 and the shell `/bin/bash`, confirming it as the primary interactive account on the machine.

```
shaonkhan@shaon:~$ getent passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:100:102:systemd Network Management,,:/run/systemd:/usr/sbin/nologin
systemd-resolve:x:101:103:systemd Resolver,,:/run/systemd:/usr/sbin/nologin
messagebus:x:102:105:/:nonexistent:/usr/sbin/nologin
systemd-timesync:x:103:106:systemd Time Synchronization,,:/run/systemd:/usr/sbin/nologin
syslog:x:104:111:/:home/syslog:/usr/sbin/nologin
_apt:x:105:65534:/:nonexistent:/usr/sbin/nologin
tss:x:106:112:TPM software stack,,:/var/lib/tpm:/bin/false
uidd:x:107:115:/:run/uidd:/usr/sbin/nologin
systemd-oom:x:108:116:systemd Userspace OOM Killer,,:/run/systemd:/usr/sbin/nologin
tcpdump:x:109:117:/:nonexistent:/usr/sbin/nologin
avahi-autoipd:x:110:119:Avahi autoip daemon,,:/var/lib/avahi-autoipd:/usr/sbin/nologin
usbmux:x:111:46:usbmux daemon,,:/var/lib/usbmux:/usr/sbin/nologin
dnsmasq:x:112:65534:dnsmasq,,:/var/lib/misc:/usr/sbin/nologin
kernoops:x:113:65534:Kernel Oops Tracking Daemon,,:/usr/sbin/nologin
avahi:x:114:121:Avahi mDNS daemon,,:/run/avahi-daemon:/usr/sbin/nologin
cups-pk-helper:x:115:122:user for cups-pk-helper service,,:/home/cups-pk-helper:/usr/sbin/nologin
rtkit:x:116:123:RealtimeKit,,:/proc:/usr/sbin/nologin
whoopie:x:117:124:/:nonexistent:/bin/false
sssd:x:118:125:SSSD system user,,:/var/lib/sss:/usr/sbin/nologin
speech-dispatcher:x:119:29:Speech Dispatcher,,:/run/speech-dispatcher:/bin/false
nm-openvpn:x:120:126:NetworkManager OpenVPN,,:/var/lib/openvpn/chroot:/usr/sbin/nologin
saned:x:121:128:/:var/lib/saned:/usr/sbin/nologin
colord:x:122:129:colord colour management daemon,,:/var/lib/colord:/usr/sbin/nologin
geoclue:x:123:130:/:var/lib/geoclue:/usr/sbin/nologin
pulse:x:124:131:PulseAudio daemon,,:/run/pulse:/usr/sbin/nologin
```

Figure 1(a): Output of `getent passwd` showing system and service accounts

```
gnome-initial-setup:x:125:65534:/:run/gnome-initial-setup:/bin/false
hplip:x:126:7:HPLIP system user,,,:/run/hplip:/bin/false
gdm:x:127:133:Gnome Display Manager:/var/lib/gdm3:/bin/false
shaonkhan:x:1002:1002:Shaon Khan,1984,01633040670,01633040670:/home/shaonkhan:/bin/bash
```

Figure 1(b): End of getent passwd output including the shaonkhan account entry

To obtain a cleaner view of usernames only, the command `compugen -u` was then executed. This shell built-in command reads the same account database but prints only the account names, one per line, without any additional fields. The output provided an uncluttered list of all accounts, making it straightforward to count and inspect them. The last entry in the `compugen -u` output was `shaonkhan`, confirming it as the most recently created human account at the time.

```
colord
geoclue
pulse
gnome-initial-setup
hplip
gdm
shaonkhan
```

Figure 1(c): `compugen -u` output showing usernames including `shaonkhan`

4.2 Creating a New User Account

The next step was to create a new user account named `sehrish`. The command `sudo adduser sehrish` was executed from the `shaonkhan` session. Since `adduser` requires root privileges, the `sudo` prefix prompted for the `shaonkhan` account's password before proceeding.

The `adduser` command then performed several operations automatically. It created a new group named `sehrish` with GID 1000 and assigned that group as the user's primary group. It created the user account `sehrish` with UID 1000. It created the home directory `/home/sehrish` and populated it by copying the skeleton files from `/etc/skel`, which typically includes default configuration files such as `.bashrc`, `.profile`, and `.bash_logout`.

The command then prompted for a password. The first password attempted was flagged by the system with the warning "BAD PASSWORD: The password fails the dictionary check - it does not contain enough DIFFERENT characters." Despite this warning, the system allowed the weak password to be accepted when the confirmation was entered, since the `adduser` tool does not enforce rejection in interactive mode. Following the password prompt, the system displayed a form asking for GECOS information. The following details were entered: Full Name as `Sehrish Khan`, Room Number as `1`, Work Phone as `01511111111`, Home Phone as `01522222222`, and the Other field was left blank. When asked "Is the information correct? [Y/n]", `Y` was entered to confirm.

```

shaonkhan@shaon:~$ sudo adduser sehrish
[sudo] password for shaonkhan:
Adding user `sehrish' ...
Adding new group `sehrish' (1000) ...
Adding new user `sehrish' (1000) with group `sehrish' ...
Creating home directory `/home/sehrish' ...
Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: The password fails the dictionary check - it does not contain enough DIFFERENT characters
Retype new password:
passwd: password updated successfully
Changing the user information for sehrish
Enter the new value, or press ENTER for the default
  Full Name []: Sehrish Khan
   Room Number []: 1
  Work Phone []: 01511111111
  Home Phone []: 01522222222
   Other []:
Is the information correct? [Y/n] Y

```

Figure 2: Creating user sehrish using sudo adduser with GECOS information

After account creation, a stronger password was set using `sudo passwd sehrish`. This command, when run with elevated privileges, allows the administrator to change any user's password without knowing the current one. A shorter test password was entered to demonstrate that the system still issues a BAD PASSWORD warning for passwords shorter than eight characters, but again proceeds to update the credential upon confirmation. The terminal displayed "passwd: password updated successfully," confirming that the credential change was applied to `/etc/shadow`.

```

shaonkhan@shaon:~$ sudo -i
root@shaon:~#
logout
shaonkhan@shaon:~$ sudo u
sudo: u: command not found
shaonkhan@shaon:~$ sudo su
root@shaon:/home/shaonkhan# cd
root@shaon:~#
exit
shaonkhan@shaon:~$ sudo passwd sehrish
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully

```

Figure 3: Resetting the password for sehrish using sudo passwd

4.3 Switching Between User Accounts

With the sehrish account created, the difference in behavior between `su username` and `su - username` was demonstrated. First, `su sehrish` was executed from within the shaonkhan home directory. The system prompted for the sehrish account's password. After authentication succeeded, a new shell was opened under the sehrish identity. However, the working directory did not change; the shell remained inside `/home/shaonkhan`, as evidenced by the prompt and subsequent behavior.

When `ls` was executed in this state, the terminal returned "ls: cannot open directory '.': Permission denied." This error occurred because the directory `/home/shaonkhan` is owned by shaonkhan and its permissions do not allow other users to read or traverse it. The sehrish account, being a different user with no special privileges over shaonkhan's home directory, was denied access. This demonstrates a

practical consequence of the non-login su behavior: the shell session is left in a directory that the new identity may not have permission to access.

When cd was executed, the shell moved to the sehrish account's home directory at /home/sehrish, and ls now returned an empty listing, confirming that the home directory existed but contained no user-created files. The working directory was confirmed to be /home/sehrish using pwd.

To return to the shaonkhan account, su shaonkhan was entered. After re-authentication, the shell returned to the shaonkhan context, now with the working directory at /home/sehrish due to where the session had been.

```
shaonkhan@shaon:~$ su sehrish
Password:
sehrish@shaon:/home/shaonkhan$ ls
ls: cannot open directory '.': Permission denied
sehrish@shaon:/home/shaonkhan$ cd
sehrish@shaon:~$ ls
sehrish@shaon:~$ pwd
/home/sehrish
sehrish@shaon:~$ su shaonkhan
Password:
shaonkhan@shaon:/home/sehrish$ cd
shaonkhan@shaon:~$
shaonkhan@shaon:~$ su - sehrish
Password:
sehrish@shaon:~$
logout
```

Figure 4: Demonstrating su sehrish behavior and Permission denied error

The su - sehrish command was then used to demonstrate the proper login shell behavior. This time, the shell immediately changed to /home/sehrish, and the terminal prompt reflected the correct home directory. The environment was fully initialized as if sehrish had logged in directly. From within this session, the mkdir Downloads command was executed to create a directory inside sehrish's home directory. Running ls confirmed that Downloads had been created successfully.

```
sehrish@shaon:~$ ls
sehrish@shaon:~$ pwd
/home/sehrish
sehrish@shaon:~$ mkdir Downloads
sehrish@shaon:~$ ls
Downloads
```

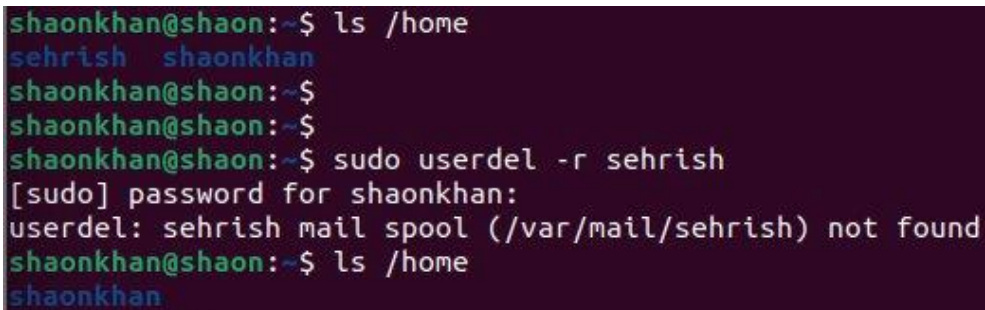
Figure 5: Creating the Downloads directory inside sehrish home directory

4.4 Deleting a User Account

After completing the session as sehrish, the logout command was used to return to the shaonkhan shell. Before deletion, `ls /home` was run to verify that both `/home/sehrish` and `/home/shaonkhan` directories were present on the filesystem.

The account was then removed using `sudo userdel -r sehrish`. The `-r` flag instructs `userdel` to recursively delete the user's home directory and mail spool in addition to removing the account entry from `/etc/passwd` and `/etc/shadow`. The system displayed a minor warning: "userdel: sehrish mail spool (`/var/mail/sehrish`) not found." This warning simply indicates that no mail spool had been created for this account during its short lifetime, which is normal for newly created accounts that have not yet received system mail. The warning is not an error and does not prevent successful account removal.

After the command completed, `ls /home` was run again. The output showed only `shaonkhan`, confirming that the `/home/sehrish` directory and all its contents, including the Downloads directory created earlier, had been removed along with the account.



```
shaonkhan@shaon:~$ ls /home
sehrish  shaonkhan
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$ sudo userdel -r sehrish
[sudo] password for shaonkhan:
userdel: sehrish mail spool (/var/mail/sehrish) not found
shaonkhan@shaon:~$ ls /home
shaonkhan
```

Figure 6: Verifying deletion of sehrish using `ls /home` before and after `userdel`

4.5 Creating a Group

The command `sudo groupadd phyra` was executed to create a new supplementary group named `phyra`. The kernel automatically assigned the next available GID to this group. To confirm that the group was created and to inspect its assigned GID, `sudo tail /etc/group` was run. This command printed the last ten lines of the group database, which included recently added entries.

The relevant portion of the output showed that the system already had several existing groups at high GID values, including `sambashare` at GID 135, `shaonkhan` at GID 1002, `Triangle_2` at GID 1004, `Triangle_1` at GID 1005, and `shayankhan` at GID 1000. The newly created `phyra` group appeared as the last entry with GID 1006, confirming successful creation. The `groups` command was also run to confirm the current user's own group memberships, which showed `shaonkhan` belonging to the `sudo` and `Triangle_2` groups in addition to the primary `shaonkhan` group.

```

shaonkhan@shaon:~$ sudo groupadd phyra
shaonkhan@shaon:~$ sudo tail /etc/group
pulse:x:131:
pulse-access:x:132:
gdm:x:133:
lxd:x:134:
sambashare:x:135:
shaonkhan:x:1002:
Triangle_2:x:1004:shaonkhan
Triangle_1:x:1005:
shayankhan:x:1000:
phyra:x:1006:
shaonkhan@shaon:~$ groups
shaonkhan sudo Triangle_2
shaonkhan@shaon:~$ groups shayankhan
shayankhan : shayankhan
shaonkhan@shaon:~$ sudo shayankhan -aG shaonkhan,phyra,Triangle_1
sudo: shayankhan: command not found
shaonkhan@shaon:~$ groups shayankhan
shayankhan : shayankhan

```

Figure 7: Group creation with groupadd phyra and verification using tail /etc/group

4.6 Creating a User and Assigning Group Memberships

Before working with group assignments, a second user account named shayankhan was created using `sudo adduser shayankhan`. The creation process followed the same interactive sequence as for sehrish: a primary group shayankhan was created with GID 1000, the home directory `/home/shayankhan` was initialized, a password was set, and GECOS information was entered with the Full Name field set to Shayan Khan and Room Number set to 1.

```

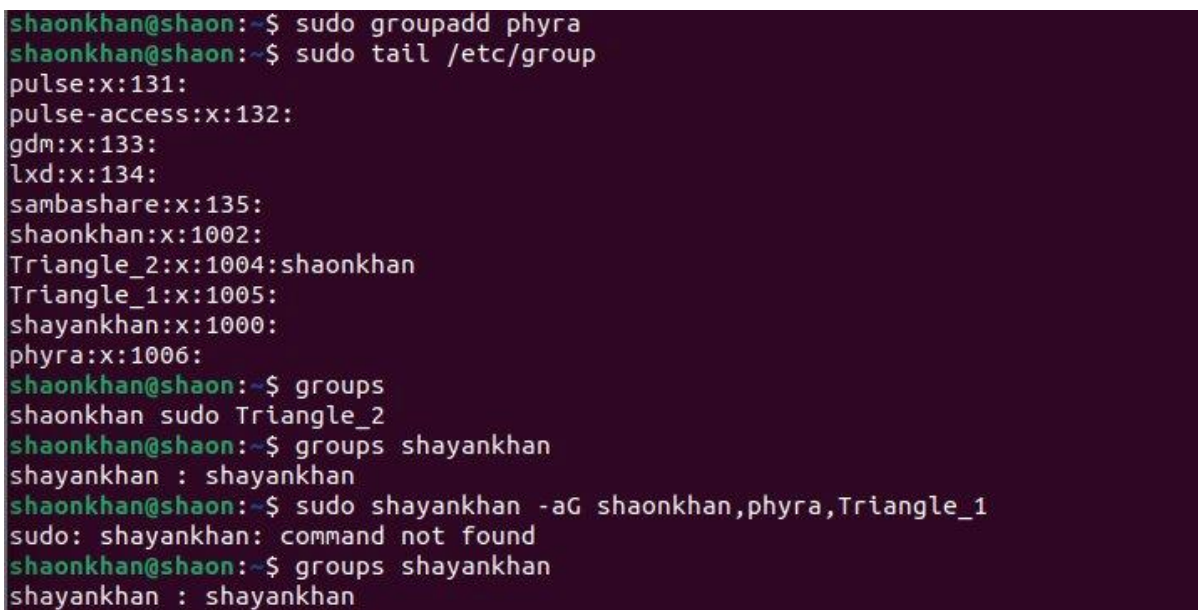
shaonkhan@shaon:~$ touch testfile.txt
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 0 May 26 09:57 testfile.txt
shaonkhan@shaon:~$ 
shaonkhan@shaon:~$ 
shaonkhan@shaon:~$ sudo adduser shayankhan
Adding user `shayankhan' ...
Adding new group `shayankhan' (1000) ...
Adding new user `shayankhan' (1000) with group `shayankhan' ...
Creating home directory `/home/shayankhan' ...
Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: The password fails the dictionary check - it does not contain enough DIFFERENT characters
Retype new password:
passwd: password updated successfully
Changing the user information for shayankhan
Enter the new value, or press ENTER for the default
    Full Name []: Shayan Khan
    Room Number []: 1
    Work Phone []: 1111111111
    Home Phone []: 1111111111
    Other []:
Is the information correct? [Y/n] Y
shaonkhan@shaon:~$ ls /homee
ls: cannot access '/homee': No such file or directory
shaonkhan@shaon:~$ ls /home
shaonkhan shayankhan
shaonkhan@shaon:~$ touch testfile.txt
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 0 May 26 09:59 testfile.txt
shaonkhan@shaon:~$ sudo chown shayankhan testfile.txt
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shayankhan shaonkhan 0 May 26 09:59 testfile.txt

```

Figure 8: Creating user shayankhan using sudo adduser

With the shayankhan account created, the next objective was to assign it membership in multiple supplementary groups: shaonkhan, phyra, Triangle_1, and Triangle_2. An error was made during the first attempt: the command entered was `sudo shayankhan -aG shaonkhan,phyra,Triangle_1`, which is syntactically incorrect. Instead of using `usermod` to invoke the user modification utility, the username shayankhan was accidentally written as the command itself. The shell interpreted shayankhan as an executable program name and returned "sudo: shayankhan: command not found." This is a common typographical error when working quickly in the terminal.

The groups shayankhan command was run after this failed attempt to confirm that no change had been made; the output showed shayankhan belonging only to its primary group, as expected.



```
shaonkhan@shaon:~$ sudo groupadd phyra
shaonkhan@shaon:~$ sudo tail /etc/group
pulse:x:131:
pulse-access:x:132:
gdm:x:133:
lxd:x:134:
sambashare:x:135:
shaonkhan:x:1002:
Triangle_2:x:1004:shaonkhan
Triangle_1:x:1005:
shayankhan:x:1000:
phyra:x:1006:
shaonkhan@shaon:~$ groups
shaonkhan sudo Triangle_2
shaonkhan@shaon:~$ groups shayankhan
shayankhan : shayankhan
shaonkhan@shaon:~$ sudo shayankhan -aG shaonkhan,phyra,Triangle_1
sudo: shayankhan: command not found
shaonkhan@shaon:~$ groups shayankhan
shayankhan : shayankhan
```

Figure 9: Incorrect command attempt and unchanged group membership of shayankhan

The corrected command `sudo usermod -aG shaonkhan,phyra,shayankhan shaonkhan` was first used to add shaonkhan itself to additional groups, and then `sudo usermod -aG shaonkhan, phyra, Triangle_1, Triangle_2 shayankhan` was executed to assign shayankhan to all four intended supplementary groups. The `id shayankhan` command was run immediately afterward to verify the result. The output confirmed:

```
uid=1000(shayankhan) gid=1000(shayankhan)
groups=1000(shayankhan),1002(shaonkhan),1004(Triangle_2),1005(Triangle_1),1006(phyra)
```

This confirmed that shayankhan was now a member of all four supplementary groups in addition to its own primary group.

```

shaonkhan@shaon:~$ sudo usermod -aG shaonkhan,phyra,shayankhan
Usage: usermod [options] LOGIN

Options:
  -b, --badnames          allow bad names
  -c, --comment COMMENT   new value of the GECOS field
  -d, --home HOME_DIR     new home directory for the user account
  -e, --expiredate EXPIRE_DATE set account expiration date to EXPIRE_DATE
  -f, --inactive INACTIVE  set password inactive after expiration
                           to INACTIVE
  -g, --gid GROUP          force use GROUP as new primary group
  -G, --groups GROUPS      new list of supplementary GROUPS
  -a, --append             append the user to the supplemental GROUPS
                           mentioned by the -G option without removing
                           the user from other groups
  -h, --help              display this help message and exit
  -l, --login NEW_LOGIN   new value of the login name
  -L, --lock              lock the user account
  -m, --move-home         move contents of the home directory to the
                           new location (use only with -d)
  -o, --non-unique         allow using duplicate (non-unique) UID
  -p, --password PASSWORD  use encrypted password for the new password
  -R, --root CHROOT_DIR   directory to chroot into
  -P, --prefix PREFIX_DIR  prefix directory where are located the /etc/* files
  -s, --shell SHELL        new login shell for the user account
  -u, --uid UID            new UID for the user account
  -U, --unlock            unlock the user account
  -v, --add-subuids FIRST-LAST add range of subordinate uids
  -V, --del-subuids FIRST-LAST remove range of subordinate uids
  -w, --add-subgids FIRST-LAST add range of subordinate gids
  -W, --del-subgids FIRST-LAST remove range of subordinate gids
  -Z, --selinux-user SEUSER new SELinux user mapping for the user account

shaonkhan@shaon:~$ id shaonkhan
uid=1002(shaonkhan) gid=1002(shaonkhan) groups=1002(shaonkhan),27(sudo),1004(Triangle_2)
shaonkhan@shaon:~$

```

Figure 10: usermod help output displayed due to missing LOGIN argument in earlier attempt

```

shaonkhan@shaon:~$ sudo usermod -aG shaonkhan,phyra,shayankhan shaonkhan
shaonkhan@shaon:~$ id shaonkhan
uid=1002(shaonkhan) gid=1002(shaonkhan) groups=1002(shaonkhan),27(sudo),1004(Triangle_2),1000(shayankhan),1006(phyra)
shaonkhan@shaon:~$ id shayankhan
uid=1000(shayankhan) gid=1000(shayankhan) groups=1000(shayankhan)
shaonkhan@shaon:~$ sudo usermod -aG shaonkhan,phyra,Triangle_1,Triangle_2 shayankhan
shaonkhan@shaon:~$ id shayankhan
uid=1000(shayankhan) gid=1000(shayankhan) groups=1000(shayankhan),1002(shaonkhan),1004(Triangle_2),1005(Triangle_1),1006(phyra)
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shayankhan shaonkhan 0 May 26 09:59 testfile.txt
shaonkhan@shaon:~$ chgrp phyra testfile.txt
chgrp: changing group of 'testfile.txt': Operation not permitted
shaonkhan@shaon:~$ sudo chgrp phyra testfile.txt
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shayankhan phyra 0 May 26 09:59 testfile.txt
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$ chmod ug+s Downloads
shaonkhan@shaon:~$ ls -l
total 36
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Desktop
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:35 Documents
drwxr-sr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Downloads
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Music
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Pictures
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Public
drwx----- 3 shaonkhan shaonkhan 4096 May 6 15:25 snap
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Templates
-rw-rw-r-- 1 shayankhan phyra 0 May 26 09:59 testfile.txt
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Videos

```

Figure 11: Successful usermod command and id verification showing all group memberships

4.7 Modifying File Ownership

A test file was created using `touch testfile.txt`. Running `ls -l testfile.txt` immediately afterward showed the file with the following attributes: `-rw-rw-r-- 1 shaonkhan shaonkhan 0 May 26 09:59 testfile.txt`. Both the owner and the group owner were shaonkhan, reflecting that the file was created in the shaonkhan session and inherited that account's primary group.

The ownership was then transferred to shayankhan using `sudo chown shayankhan testfile.txt`. Running `ls -l` again confirmed the change: `-rw-rw-r-- 1 shayankhan shaonkhan 0 May 26 09:59 testfile.txt`. The user owner field now showed shayankhan while the group owner remained shaonkhan, demonstrating that `chown` in this form affects only the user owner and leaves the group field unchanged.

```
shaonkhan@shaon:~$ touch testfile.txt
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 0 May 26 09:57 testfile.txt
shaonkhan@shaon:~$ 
shaonkhan@shaon:~$ 
shaonkhan@shaon:~$ sudo adduser shayankhan
Adding user `shayankhan' ...
Adding new group `shayankhan' (1000) ...
Adding new user `shayankhan' (1000) with group `shayankhan' ...
Creating home directory `/home/shayankhan' ...
Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: The password fails the dictionary check - it does not contain enough DIFFERENT characters
Retype new password:
passwd: password updated successfully
Changing the user information for shayankhan
Enter the new value, or press ENTER for the default
  Full Name []: Shayan Khan
   Room Number []: 1
   Work Phone []: 1111111111
   Home Phone []: 1111111111
    Other []:
Is the information correct? [Y/n] Y
shaonkhan@shaon:~$ ls /homee
ls: cannot access '/homee': No such file or directory
shaonkhan@shaon:~$ ls /home
shaonkhan  shayankhan
shaonkhan@shaon:~$ touch testfile.txt
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 0 May 26 09:59 testfile.txt
shaonkhan@shaon:~$ sudo chown shayankhan testfile.txt
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shayankhan shaonkhan 0 May 26 09:59 testfile.txt
```

Figure 12: File creation, ownership inspection, and `chown` applied to `testfile.txt`

4.8 Modifying File Group Ownership

The next step was to change the group ownership of `testfile.txt` from shaonkhan to phyra. The first attempt used the command `chgrp phyra testfile.txt` without `sudo`. The terminal returned the error "chgrp: changing group of 'testfile.txt': Operation not permitted." This error occurred because at this point in the session, the file's user owner was shayankhan, not shaonkhan. Since shaonkhan does not own the file, it cannot modify its group, even if it belongs to the target group phyra. Linux enforces the rule that only the file's owner can change its group, and only to a group the owner belongs to. A non-owner cannot perform this operation without root privileges.

The operation was then retried using `sudo chgrp phyra testfile.txt`. With elevated privileges, the command succeeded. Running `ls -l` again confirmed the change: `-rw-rw-r-- 1 shayankhan phyra 0 May 26 09:59 testfile.txt`. The group owner field now correctly showed `phyra`.

```
shaonkhan@shaon:~$ sudo usermod -aG shaonkhan,phyra,shayankhan shaonkhan
shaonkhan@shaon:~$ id shaonkhan
uid=1002(shaonkhan) gid=1002(shaonkhan) groups=1002(shaonkhan),27(sudo),1004(Triangle_2),1000(shayankhan),1006(phyra)
shaonkhan@shaon:~$ id shayankhan
uid=1000(shayankhan) gid=1000(shayankhan) groups=1000(shayankhan)
shaonkhan@shaon:~$ sudo usermod -aG shaonkhan,phyra,Triangle_1,Triangle_2 shayankhan
shaonkhan@shaon:~$ id shayankhan
uid=1000(shayankhan) gid=1000(shayankhan) groups=1000(shayankhan),1002(shaonkhan),1004(Triangle_2),1005(Triangle_1),1006(phyra)
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shayankhan shaonkhan 0 May 26 09:59 testfile.txt
shaonkhan@shaon:~$ chgrp phyra testfile.txt
chgrp: changing group of 'testfile.txt': Operation not permitted
shaonkhan@shaon:~$ sudo chgrp phyra testfile.txt
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shayankhan phyra 0 May 26 09:59 testfile.txt
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$ chmod ug+s Downloads
shaonkhan@shaon:~$ ls -l
total 36
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Desktop
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:35 Documents
drwsr-sr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Downloads
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Music
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Pictures
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Public
drwxr-xr-x 3 shaonkhan shaonkhan 4096 May 6 15:25 snap
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Templates
-rw-rw-r-- 1 shayankhan phyra 0 May 26 09:59 testfile.txt
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Videos
```

Figure 13: `chgrp` failure without `sudo` and successful execution with `sudo`, followed by `ls -l` verification

4.9 Applying Special Permission Bits (SGID)

The final step of the experiment involved applying the SGID permission bit to the Downloads directory, which was located in the shaonkhan home directory. The command `chmod ug+s Downloads` was executed. This applies both the SUID (`u+s`) and SGID (`g+s`) bits to the Downloads directory.

Following this, `ls -l` was executed to display the permission strings for all entries in the home directory. The line corresponding to Downloads read: `drwsr-sr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Downloads`. In this permission string, the third character of the owner permission triplet (normally `x` for execute) had changed to `s`, and similarly the third character of the group permission triplet had changed to `s`. These two `s` characters indicate that both the SUID and SGID bits are active on the directory. For the Downloads directory, the SGID effect is particularly meaningful: any file created inside Downloads by any user will inherit the group ownership of the directory itself rather than the creating user's primary group. This ensures consistent group ownership across all files placed in the directory, which is essential for collaborative access patterns.

```

shaonkhan@shaon:~$ sudo usermod -aG shaonkhan,phyra,shayankhan shaonkhan
shaonkhan@shaon:~$ id shaonkhan
uid=1002(shaonkhan) gid=1002(shaonkhan) groups=1002(shaonkhan),27(sudo),1004(Triangle_2),1000(shayankhan),1006(phyra)
shaonkhan@shaon:~$ id shayankhan
uid=1000(shayankhan) gid=1000(shayankhan) groups=1000(shayankhan)
shaonkhan@shaon:~$ sudo usermod -aG shaonkhan,phyra,Triangle_1,Triangle_2 shayankhan
shaonkhan@shaon:~$ id shayankhan
uid=1000(shayankhan) gid=1000(shayankhan) groups=1000(shayankhan),1002(shaonkhan),1004(Triangle_2),1005(Triangle_1),1006(phyra)
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shayankhan shaonkhan 0 May 26 09:59 testfile.txt
shaonkhan@shaon:~$ chgrp phyra testfile.txt
chgrp: changing group of 'testfile.txt': Operation not permitted
shaonkhan@shaon:~$ sudo chgrp phyra testfile.txt
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shayankhan phyra 0 May 26 09:59 testfile.txt
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$ chmod ug+s Downloads
shaonkhan@shaon:~$ ls -l
total 36
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Desktop
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:35 Documents
drwsr-sr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Downloads
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Music
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Pictures
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Public
drwx----- 3 shaonkhan shaonkhan 4096 May 6 15:25 snap
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Templates
-rw-rw-r-- 1 shayankhan phyra 0 May 26 09:59 testfile.txt
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Videos

```

Figure 14: Permission string showing drwsr-sr-x after chmod ug+s applied to Downloads

5. Discussion and Analysis

Several observations from the experiment merit detailed discussion, as they reveal important aspects of how Linux manages users, groups, and permissions in practice.

5.1 Command Syntax Errors and the usermod Help Menu

During Step 4.6, the accidental command `sudo shayankhan -aG shaonkhan,phyra,Triangle_1` caused `sudo` to interpret the username `shayankhan` as the executable to run. Since no program by that name exists in the system `PATH`, `sudo` returned "command not found." This illustrates a fundamental Linux behavior: the shell looks up the first token of a command in the directories listed in `PATH`. If it is not found, the command fails immediately, regardless of any flags that follow.

In a related event captured in the terminal, an incomplete `usermod` call was attempted without specifying the `LOGIN` argument at the end. When `usermod` receives no username argument, it cannot determine which account to modify and therefore prints its full help and usage documentation to standard error before exiting. This behavior is intentional: well-written UNIX utilities display usage information when called incorrectly, providing immediate guidance on proper syntax without requiring the user to consult a separate manual page. The verbose help output for `usermod` also serves as a useful reference for all available options, including account expiration, home directory relocation, shell assignment, and UID modification.

5.2 Ownership Constraints on `chgrp`

The failure of the unprivileged `chgrp phyra testfile.txt` command in Step 4.8 is a direct expression of one of Linux's foundational security principles. The file's user owner was `shayankhan` at the time of the attempt, having been changed in Step 4.7. The active session was running as `shaonkhan`, meaning `shaonkhan` was not the file owner. Linux explicitly prohibits non-owners from changing the group ownership of a file, even if the non-owner is a member of the target group and would benefit from the change. This restriction prevents one user from silently re-associating another user's files with groups the owner did not intend.

Even the file owner, when changing group ownership without root privilege, is restricted: the owner can only change the group to one of the groups to which the owner personally belongs. This prevents a file owner from assigning their file to a group that the owner does not have membership in, which could be used as a method to make the file accessible to a group the owner should not interact with. These layered restrictions collectively protect the integrity of the group-based access control model.

5.3 Security and Collaboration Benefits of SGID Directories

The application of the SGID bit to the Downloads directory in Step 4.9 illustrates an important tool for managing shared workspaces. In a typical Linux environment without SGID, when multiple users belonging to a common group write files to the same directory, each file inherits the primary group of its creator rather than any shared group. If users have different primary groups, the resulting files will have different group owners, making it difficult for other group members to access them uniformly.

With SGID enabled on a shared directory, all files created within it automatically inherit the group ownership of the directory itself. If the directory is owned by a shared project group, every file written into it becomes accessible to all members of that group regardless of who created it. This removes the need for each contributor to manually change the group ownership of every file they create, which is both inconvenient and prone to oversight. SGID directories are therefore widely used in multi-user environments such as project directories, shared research repositories, and administrative workspace folders.

5.4 Privilege Separation and Administrative Security

Throughout the experiment, the sudo mechanism was consistently required to perform administrative operations such as creating users, deleting accounts, creating groups, assigning supplementary group memberships, and changing file ownership. This pattern reflects the principle of least privilege, which holds that a user or process should operate with only the minimum level of access required to complete its current task. By restricting administrative commands to privileged execution, Linux limits the potential damage caused by mistakes or misuse in an ordinary session.

The separation of `/etc/passwd` and `/etc/shadow` further supports this principle. An ordinary program reading account information to map UIDs to names only needs access to `/etc/passwd`, which is world-readable. No ordinary process requires access to the password hashes, and restricting `/etc/shadow` to root prevents credential theft by unprivileged programs or users. The combination of shadow passwords, UID-based permission enforcement, and group-based access control forms a layered security architecture that makes Linux suitable for multi-user deployment in institutional environments.

6. Conclusion

This laboratory session provided comprehensive hands-on experience with the user and group management subsystem of Linux. Through a series of structured exercises, the complete lifecycle of a user account was observed and practiced: account creation using `adduser`, password management using `passwd`, interactive session switching using `su` and `su -`, and account removal using `userdel -r`. Each step produced observable system behavior that reinforced the underlying theory.

The distinction between `su` and `su -` proved practically significant. The non-login form of `su` left the shell in the calling user's directory, which resulted in a Permission denied error when the new user attempted to list that directory's contents, since they lacked the necessary permissions. The login form `su -` correctly initialized the environment by changing to the target user's home directory and loading appropriate shell configuration files.

Group management tasks demonstrated how Linux organizes users into access control units. The creation of the `phyra` group, verified through `tail /etc/group`, showed how GIDs are assigned sequentially. The assignment of `shayankhan` to multiple supplementary groups using `usermod -aG`, confirmed by `id shayankhan`, illustrated how a single account can access resources owned by several different groups simultaneously.

The file ownership exercises clarified the distinction between user ownership and group ownership, and the asymmetric privilege requirements for each. While `chown` requires root privilege to transfer ownership to another user, `chgrp` by an ordinary user is constrained to files they own and groups they belong to. The failure of the unprivileged `chgrp` command, followed by its success under `sudo`, illustrated these constraints directly.

Finally, the application of the SGID permission bit using `chmod ug+s`, and its verification through the `s` characters visible in `ls -l` output, demonstrated a practical mechanism for enforcing consistent group ownership in shared directories. The session concluded with a clear understanding of how special permission bits extend the basic read-write-execute model to support more sophisticated access control patterns suited to collaborative and multi-user environments.

Overall, the experiment confirmed that Linux's approach to user, group, and permission management is both precise and flexible, enforcing strict security boundaries while providing the administrative tools necessary to configure appropriate access for any organizational requirement.